

Vayu: Architecture and Data Model

Supporting document, SIH 2026, ByteBrigade. Describes the prototype as implemented in the repository.

1. Components

Component	Runs on	Role
Scraper and index pipeline (Python, Scrapling, headless Chromium)	GitHub Actions (ubuntu runner)	Collects fares, normalises, filters anomalies, computes the index, writes to the database. Needs a real browser and long run time, which serverless functions cannot provide.
Database (PostgreSQL)	Supabase	Shared state: daily_index, route_daily_detail, scrape_runs, daily_index_review.
Dashboard (Next.js, React, SWR, Recharts)	Vercel	Reads the database and renders the index, chart and 325-route table. 'Refresh' re-reads the database; 'Run live scrape' triggers the workflow through the GitHub API.

2. Data flow

Scheduled trigger (03:17 UTC daily, full 325-route basket) or dashboard button (quick run, top 20 routes by weight) starts the GitHub Actions workflow. The workflow runs: scrape, normalise, anomaly filter, index computation, then upserts to Supabase using a server-side service key. The dashboard reads the same tables with public read-only access and polls scrape_runs to show run status.

3. Data model

Table	Key	Main columns
daily_index	observation_date	index_value, base_date
route_daily_detail	route_id + observation_date	weight, route_price, price_relative, source ('scraped' or 'sample'), airlines[], origin_name, destination_name
scrape_runs	id	started_at, finished_at, target, mode (quick/full), routes_attempted, routes_scraped, routes_anomalous, coverage_weight_pct, status (running/completed/failed), error
daily_index_review	observation_date	reviewed_at (analyst sign-off marker)

Writes are upserts keyed on date (and route plus date), so re-running a day replaces that day's row. Each run still adds a new row to scrape_runs, which keeps an audit trail of every run.

4. Dashboard API routes

Route	Method	Purpose
/api/index-data	GET	Latest index, history, and the route table for the latest observation date
/api/rescrape	POST	Dispatches the quick-mode workflow through the GitHub API
/api/rescrape/status	GET	Reports whether the triggered run has finished (reads scrape_runs)
/api/review	POST	Records an analyst sign-off for a day's index

5. Security posture

Row-level security is enabled on all tables. Public users can read only; there is no public insert, update or delete policy. Only the server-side service key can write, and it is stored as an environment secret in Vercel and GitHub Actions, never sent to the browser.

Current gaps: the /api/review and /api/rescrape endpoints do not yet authenticate the caller or rate-limit requests, so they are suitable for a demo, not for public production use. Authentication and rate limiting are required before any non-demo deployment.

6. Failure handling

A failed pipeline run is recorded in `scrape_runs` with status 'failed' and the error text, so failures are visible in the database and not only in the CI log. Routes with no fresh trusted data are shown as 'Sample / last known' and excluded from the index sum.